

From Frontend to Backend with FastAPI: Your Complete Guide to APIs

14 min read · Oct 15, 2023



Vaishnav Manoj



Listen



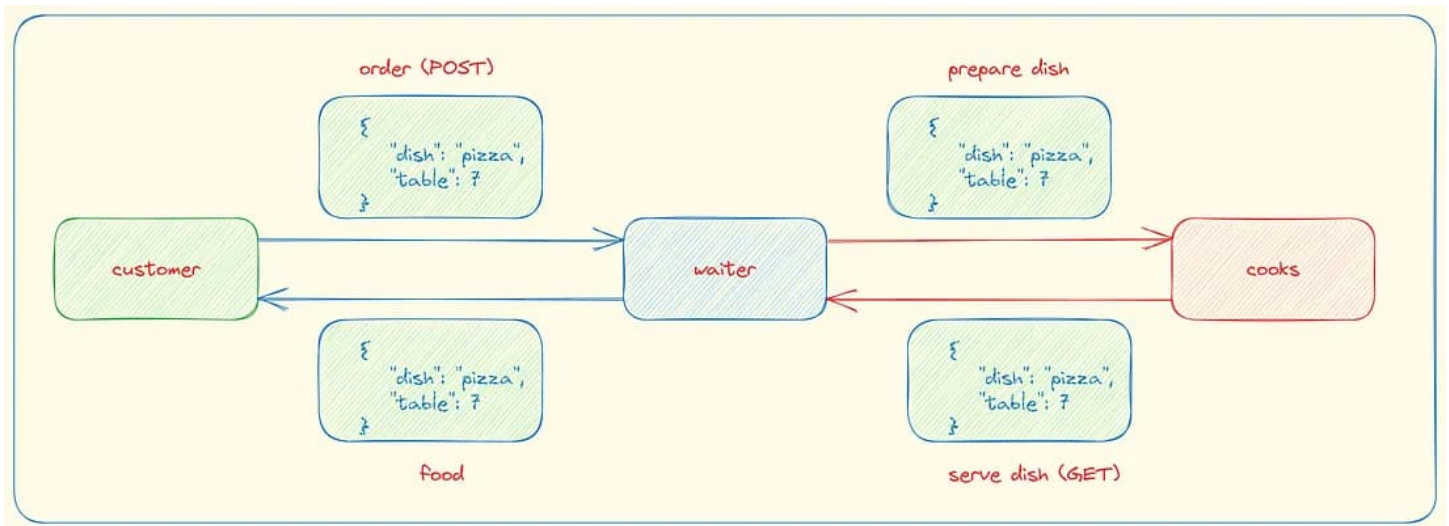
Share

APIs
101

What are APIs in simple terms?

Imagine you're at a restaurant, and you're hungry for some delicious food. You sit down at your table and eagerly await the menu. In this scenario, the menu is like an API (Application Programming Interface).

1. **Menu as an API:** The menu is your interface to the restaurant's kitchen. It doesn't show you the kitchen's secrets, but it presents the dishes you can order and the information you need to make your choice.
2. **Ordering:** When you decide what you want, you don't need to enter the kitchen and cook the meal yourself. Instead, you communicate your choice to the waiter (the intermediary), who conveys your request to the kitchen.



example analogy of how APIs work

- **Customer:** Represents a software or application that wants to request information or services.
- **Waiter:** Acts as the intermediary, similar to an API, which takes the customer's request and conveys it to the restaurant (kitchen).
- **Restaurant (Kitchen):** Represents the system or application that has the data or services the customer wants.

In this analogy, the customer (software) interacts with the waiter (API) to request a specific service (dish) from the restaurant (system). The API request is analogous to placing an order, and the API response is like receiving the prepared dish (data or service) in return.

Where are APIs used?

Let's explore some simple and beginner-friendly use cases of APIs. These examples will help you grasp how APIs are commonly used in everyday scenarios:

1. **Weather Information:** Imagine you're using a weather app on your phone. The app doesn't generate weather data itself; instead, it fetches weather information from a weather service's API. This API provides real-time weather updates that the app displays.
2. **Map Directions:** When you're navigating using a mapping app like Google Maps, it uses APIs to fetch directions and real-time traffic data. You input your destination, and the app queries an API to calculate the best route for you.
3. **Social Media Sharing:** Have you ever shared a news article or a photo on social media? These platforms often allow third-party applications to connect via APIs. When you share something from an app, it communicates with the social media platform's API to post your content.
4. **Chat Applications:** Messaging apps use APIs to send and receive messages. When you send a text or an emoji in a chat app, it's using an API to transmit that data to the recipient.

Using an API

To illustrate how developers interact with APIs, let's go through a real-world example of calling an actual API. We'll start with a detailed diagram of the client, server, API, and database components, and then explain each component in great detail, including endpoints, requests, and responses.

The REST API url we are using for this example is:

```
https://api.sampleapis.com/coffee/hot
```

Components Explained:

1. **Client:** The client, which can be a web browser or a mobile app, initiates the process by sending an HTTP request to the API.
2. **Server:** The server hosts the API, receives the client's request, and processes it.
3. **API:** This is the core of the process. It receives the client's request and serves as an intermediary between the client and the database.
4. **Database:** The database stores the data the API needs to fulfill the request. In this case, it holds information about different types of coffee.

Flow of Data:

1. The client's HTTP GET request is sent to the server.
2. The server forwards the request to the API.
3. The API processes the request, including identifying the endpoint, and queries the database.
4. The database responds to the API with the requested data.
5. The API sends an HTTP response back to the client with the data.

Understanding the URL

Let's break down the URL "<https://api.sampleapis.com/coffee/hot>" step by step:

1. **Protocol (https):** This specifies the protocol used for communication, which, in this case, is "**https**," indicating a secure HTTP connection. It ensures that data transmitted between the client and the server is encrypted for security.
2. **Domain (api.sampleapis.com):** The domain is the address of the server that hosts the API. In this URL, "**api.sampleapis.com**" is the domain. It's like the **postal address** for the server on the internet.

3. **Path (/coffee/hot):** The path is a specific route or location on the server where the API is hosted. In our URL, “/coffee/hot” is the path. It’s similar to navigating to a specific folder or directory on a computer.
4. **API Endpoint:** The complete URL, including the domain and path, specifies a unique API endpoint. It’s the specific location where the API will respond to requests. In this case, “<https://api.sampleapis.com/coffee/hot>” is the endpoint.

So, when a client (e.g., a web browser or an application) wants to interact with this API, it forms a request with this URL, specifying the protocol, domain, and endpoint. In this example, it’s like telling the browser to go to a specific secure address (<https://api.sampleapis.com>) and ask for information related to “hot coffee” (the endpoint). The server hosting the API processes this request and provides the relevant data in response.

Understanding HTTP Requests and Responses

HTTP stands for **Hypertext Transfer Protocol**, which is the foundation of data communication on the World Wide Web.

- An HTTP request is a message sent by a client (e.g., a web browser, a mobile app, or any software) to request information from a server.
- It consists of **Requests and Responses**. Request is the data required to make the API call and Response is the data you get in return.

HTTP Requests

It consists of several parts:

Request Method:

This specifies the action the client wants the server to perform. Common methods include:

1. GET Request:

- The GET method is used to retrieve data from a specified resource.
- It’s a **read-only** request, meaning it doesn’t modify data on the server.
- Commonly used in web browsers when you visit a website, as it requests web pages and assets (images, styles, scripts).
- GET requests should not have a request body and are typically cached by browsers for faster retrieval.

2. POST Request:

- The POST method is used to submit data to be processed to a specified resource.
- It often **creates a new resource** on the server or performs an action like submitting a form or making a transaction.
- POST requests can have a request body containing data to be processed by the server.

3. PUT Request:

- The PUT method is used to **update or replace** an existing resource or create a new resource if it doesn't exist.
- It's **idempotent**, meaning repeated requests will have the same result, ensuring that making multiple identical requests doesn't cause unexpected behavior.
- A PUT request typically requires sending the entire resource representation for the update, meaning if you want to update a single field, you need to **send the entire resource** with the modification.

4. DELETE Request:

- The DELETE method is used to request the **removal of a specified resource**.
- It deletes the resource and its associated data from the server.
- DELETE requests usually **do not have a request body**.

5. PATCH Request:

- The PATCH method is used to apply **partial modifications** to a resource.
- It's often used when you only want to **update specific fields** or attributes of a resource, without affecting the entire resource.
- PATCH requests require a request body that specifies the changes to be made.

Difference between PUT and PATCH:

The main difference lies in how they handle updates:

- **PUT:** It replaces the **entire resource with the new representation**. If you omit a field in the new representation, it's often interpreted as empty or null, potentially overwriting existing data. It's not well-suited for partial updates.
- **PATCH:** It updates **specific fields of the resource**. You send only the changes you want to apply, making it suitable for partial updates without affecting other

fields. It's more efficient when you need to make small modifications.

URL:

This is the address of the resource or API endpoint the client wants to interact with.

Headers:

These provide additional information about the request, such as the type of data the client can accept or authentication credentials.

Body:

For methods like POST and PUT, the body contains data that the client wants to send to the server, often in JSON format.

```
import requests

# Define the API endpoint URL
url = "https://api.sampleapis.com/coffee/hot"

# Send an HTTP GET request
response = requests.get(url)

# Check if the request was successful (status code 200)
if response.status_code == 200:
    data = response.json()
    print("Data received:")
    print(data)
else:
    print(f"Request failed with status code: {response.status_code}")
```

HTTP Responses

An HTTP response is a message sent by a server to the client in reply to an HTTP request.

It also consists of several parts:

1. Status Codes

This is a **three-digit number** that indicates the outcome of the request. Common status codes include:

- **200 OK:** The request was successful, and the server is sending the requested data.

- **201 Created:** The request was successful, and the server has created a new resource.
- **400 Bad Request:** The request was malformed or invalid.
- **404 Not Found:** The requested resource was not found on the server.
- **500 Internal Server Error:** An error occurred on the server while processing the request.

2. Headers:

Similar to request headers, these provide additional information about the response, such as the type of data being sent or server information.

3. Body:

This contains the data or content sent by the server in response to the client's request. The data can be in various formats, such as HTML, JSON, XML, or plain text.

```
[  
  {  
    "title": "Black",
```

```

        "description": "Black coffee is as simple as it gets with ground coffee",
        "ingredients": [
            "Coffee"
        ],
        "image": "https://upload.wikimedia.org/wikipedia/commons/thumb/4/45/A_s",
        "id": 1
    },
    {
        "title": "Latte",
        "description": "As the most popular coffee drink out there, the latte i",
        "ingredients": [
            "Espresso",
            "Steamed Milk"
        ],
        "image": "https://upload.wikimedia.org/wikipedia/commons/thumb/9/9f/Lat",
        "id": 2
    },
    ...
]

```

FastAPI: Modern Python Framework for Building APIs

FastAPI is a cutting-edge Python web framework designed for building APIs with speed and ease. It has gained significant popularity in the developer community due to its exceptional features and advantages. Let's take a closer look at what makes FastAPI stand out:

Automatic Documentation:

FastAPI simplifies API development by automatically generating interactive and user-friendly documentation. This means you don't need to separately document your API endpoints. It's a developer-friendly feature, making collaboration and integration smoother.

Type Safety and Validation:

FastAPI leverages Python type hints to bring type safety to API development. This not only helps catch errors early but also assists in auto-generating accurate API schemas. You get validation and serialization of request and response data without the need for extra code.

Async Support:

FastAPI fully supports asynchronous programming, which is a game-changer for high-performance APIs. You can create asynchronous routes and utilize Python's `async` and `await` to handle concurrent requests efficiently.

Creating your own FastAPI project

GitHub - vaishnav-mk/fastapi-tutorial: 📖 Basic tutorial on what fastapi is

📖 Basic tutorial on what fastapi is. Contribute to vaishnav-mk/fastapi-tutorial development by creating an account on...

github.com

Installation:

- First, ensure you have Python installed on your system.
- Install FastAPI and Uvicorn, a lightweight ASGI server, using pip:

```
pip install fastapi uvicorn
```

Code Setup:

- Create a new Python file (e.g., `main.py`) to write your FastAPI application.
- Import the FastAPI class from the `fastapi` module:

```
from fastapi import FastAPI
app = FastAPI()
```

Define Endpoints:

- Define your API endpoints using FastAPI decorators.
- Add routes to your application using functions. For example:

```
@app.get("/")
def read_root():
    return {"Hello": "World"}
```

Run Your Application:

- To run your FastAPI application, use Uvicorn with the following command:

```
uvicorn main:app --reload
```

Here, `main` refers to the name of your Python file (without the `.py` extension), and `app` is the FastAPI instance you created.

Access Your API:

- Your FastAPI application will be accessible at `http://localhost:8000` by default.
- You can use tools like `curl` or a web browser to access your defined endpoints.

Accessing Swagger UI

Swagger UI is a powerful built-in feature of FastAPI that simplifies API documentation. By accessing `localhost:8000/docs` on your FastAPI application, you'll discover an interactive web interface that provides detailed information about your API endpoints. With Swagger UI, you can explore and test your API, making it easier to understand and interact with your API's capabilities.

GET / route

PUT /items/:item_id route

DELETE /items/:item_id route

MVC (Model-View-Controller) Architecture

[Model-View-Controller \(MVC\) Explained — With Legos — Real Python](#)

MVC is a popular software architectural pattern used for organizing code in applications, particularly in web and desktop applications. It separates an application into three interconnected components:

Model (Data):

- The Model represents the application's data and business logic.
- It manages and maintains the state and behavior of the application.
- It is responsible for data storage, retrieval, validation, and manipulation.
- Changes in the Model directly impact the View.

View (Presentation):

- The View is responsible for presenting the data to the user.
- It deals with the user interface, including elements like buttons, forms, and graphical components.
- The View doesn't handle data directly but instead receives it from the Model.

- It's responsible for displaying data and responding to user interactions.

Controller (Logic):

- The Controller acts as an intermediary between the Model and the View.
- It receives user input and translates it into actions for the Model or the View.
- It contains application logic, including how data is processed and presented.
- It updates the Model based on user interactions and can trigger updates in the View.

Testing APIs

When it comes to testing APIs, developers have various tools at their disposal. Two popular options include:

1. **cURL:** This command-line tool is a developer's Swiss army knife for making HTTP requests to test APIs. With cURL, you can send **GET**, **POST**, **PUT**, **DELETE**, and other requests to interact with your API directly from the terminal. It's a lightweight and versatile tool for quick API testing and debugging.

cURL example

2. **Postman:** Postman is a user-friendly API testing tool with a graphical interface. It allows you to create and organize API requests, manage environments, and automate testing. Postman is particularly useful for testing complex workflows, APIs with authentication, and for sharing test collections with team members.

Postman example

Both cURL and Postman serve different needs: cURL is great for quick tests and scripting, while Postman provides a comprehensive environment for in-depth API testing and collaboration. The choice between them often depends on the complexity of your testing requirements and your preferred workflow.

Security in APIs

When it comes to securing APIs, several crucial aspects need attention to protect your digital assets effectively:

Authentication:

- Authentication is the process of **verifying the identity of users** or systems interacting with your API.
- Common authentication methods include **API keys**, **tokens** (such as JSON Web Tokens or OAuth tokens), and basic authentication.
- Effective authentication ensures that only authorized users or systems can access your API.

Authorization:

- Authorization goes beyond authentication and determines what actions users or systems are allowed to perform once they've gained access.

- **Role-based access control** (RBAC) and permissions are often used to manage who can read, write, or modify data through your API.
- Proper authorization ensures that users have the right level of access without overstepping their boundaries.

Rate Limiting:

- Rate limiting helps prevent abuse of your API by limiting the number of requests a user or system can make within a defined time frame.
- It's crucial for protecting your API from distributed denial-of-service (DDoS) attacks and ensuring fair usage.
- Rate limits can be defined based on **IP addresses, user accounts, or API keys**.

HTTPS and Encryption:

- To protect data in transit, always use HTTPS to encrypt communication between clients and your API.
- Encryption ensures that data exchanged between the client and the server remains **confidential** and **secure**.

Input Validation and Sanitization:

- Validate and sanitize input data to prevent injection attacks, such as SQL injection and cross-site scripting (XSS).
- This ensures that malicious data can't compromise the integrity of your API or your database.

API Security Best Practices:

OWASP API Security Project

OWASP API Security Project on the main website for The OWASP Foundation. OWASP is a nonprofit foundation that works to...

[owasp.org\](https://owasp.org/)

- Follow established API security best practices, such as keeping your software and libraries up to date to patch known vulnerabilities.
- Implement proper error handling to avoid leaking sensitive information in error responses.

- Protect against brute force attacks by implementing account lockout mechanisms.

Throttling and Monitoring:

- Implement monitoring solutions to keep an eye on your API's performance and security.
- Set up automated alerts to detect and respond to unusual activity.
- Implement throttling to limit the number of requests from a single client in a short period, which can help mitigate abuse and protect your resources.

The Future of APIs

The API landscape is poised for a transformative journey, with cutting-edge technologies like [gRPC](#), [WebSockets](#), and [GraphQL](#) reshaping data exchange. As we look ahead, expect:

- **[Efficiency Through gRPC](#):** gRPC's high-performance capabilities will revolutionize real-time communication and microservices architecture.
- **[Real-Time Experiences](#):** WebSockets will bring unparalleled real-time interactivity to applications, from gaming to collaborative tools.
- **[Flexible Data Access](#):** GraphQL will empower clients to request only the data they need, enhancing API efficiency and user experiences.

For deeper insights into the future of APIs, check out this blog by me:

[Revolutionizing Scalability: How Microservices and gRPC Are Changing the Game](#). Additionally, explore [Cloudflare Workers: Empowering Developers to Build Faster, Scalable, and Reliable Applications](#) for further reading on the API technologies of tomorrow.

Revolutionizing Scalability: How Microservices and gRPC are Changing the Game

This article discusses how microservices and gRPC are changing the way software development approaches scalability.

medium.com

Cloudflare Workers: Empowering Developers to Build Faster, Scalable, and Reliable Applications

Further Reading & References

Here are some references and further reading materials for various aspects related to APIs:

API Basics:

1. [Wikipedia: Application Programming Interface \(API\)](#) — A comprehensive overview of APIs.
2. [MDN Web Docs: What is an API?](#) — A beginner-friendly explanation of APIs.

RESTful APIs:

1. [Roy Fielding's Dissertation on REST](#) — The original dissertation by Roy Fielding, the creator of REST, explaining the principles of RESTful architecture.

gRPC:

1. [gRPC Official Documentation](#) — The official documentation for gRPC, including tutorials and examples.
2. [gRPC: Up and Running](#) — Book by [Kasun Indrasiri](#) and [Danesh Kuruppu](#) — A comprehensive guide to gRPC.

WebSockets:

1. [MDN WebSockets API](#) — A detailed reference for using WebSockets in web development.
2. [WebSocket.org](#) — A central resource for WebSocket-related information.

GraphQL:

1. [Official GraphQL Documentation](#) — A great starting point to understand GraphQL and how to use it.
2. [How To GraphQL](#) — A comprehensive resource with tutorials and examples for learning GraphQL.

API Security:

1. [OWASP API Security Top Ten](#) — A list of the top ten API security threats and best practices for mitigating them.

2. [API Security Best Practices](#) — A guide to best practices in API security.

Serverless APIs:

1. [AWS Lambda Documentation](#) — The official documentation for AWS Lambda, a popular serverless platform.
2. [Cloudflare Workers®](#) — Deploy serverless code instantly across the globe to give it exceptional performance, reliability, and scale.

To learn more about me and discover additional insights on web development, cloud computing, and serverless architectures, visit <https://vaishnav.one>. You can also explore my Medium articles by visiting <https://wishee.medium.com> for more in-depth content or connect with me on Twitter @ https://twitter.com/vaishnav_mk1

API

Fastapi

Programming

Json

Best Practices



Published in DataX Journal

699 followers · Last published Feb 25, 2026

This is the publication of the Data Science Community, a data science-based student-led innovation community at SRM IST. Here we publish blogs based on Data Analytics, Machine Learning, web and app development, current affairs in technology and more based on experience and work



Written by Vaishnav Manoj

7.5K followers · 11 following

Pushing the boundaries of what I know to create weird and wonderful projects!

Responses (3)



See all responses